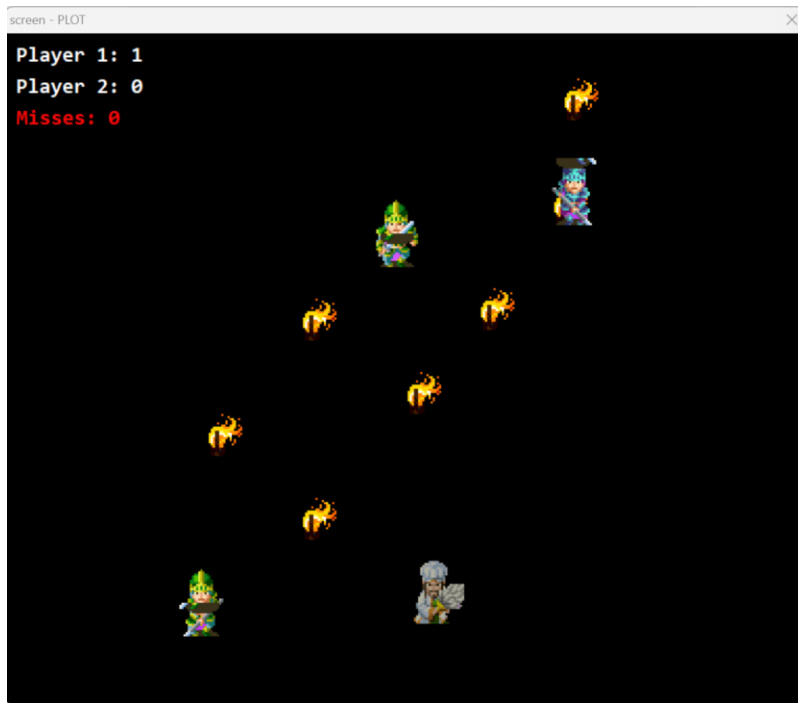


Background

In 234 AD, Shu Han chancellor Zhuge Liang died during his fifth expedition against the Cao Wei dynasty. Now, almost 2000 years later, many still wanted him to succeed. In the novel *Romance of Three Kingdoms*, Zhuge Liang arranged seven lamps in a Big Dipper (Ursa Major) formation. He stated that should the lamps continue to burn for seven days, he could live for another twelve years. On the seventh day, Cao Wei forces attempted a raid on the Shu Han camp. When Shu General Wei Yan rushed to report the raid, he allowed wind to pour in, extinguishing the lamps. Zhuge Liang soon died.

Gameplay

In this game, the two players must defend the lamps from incoming Wei soldiers. The game functions similar to whack-a-mole. The easter egg of this game can be achieved after one of the players interacts with Zhuge Liang. The game is lost when 7 enemies go undefeated. The game is won when the players successfully defeat 49 enemies combined.



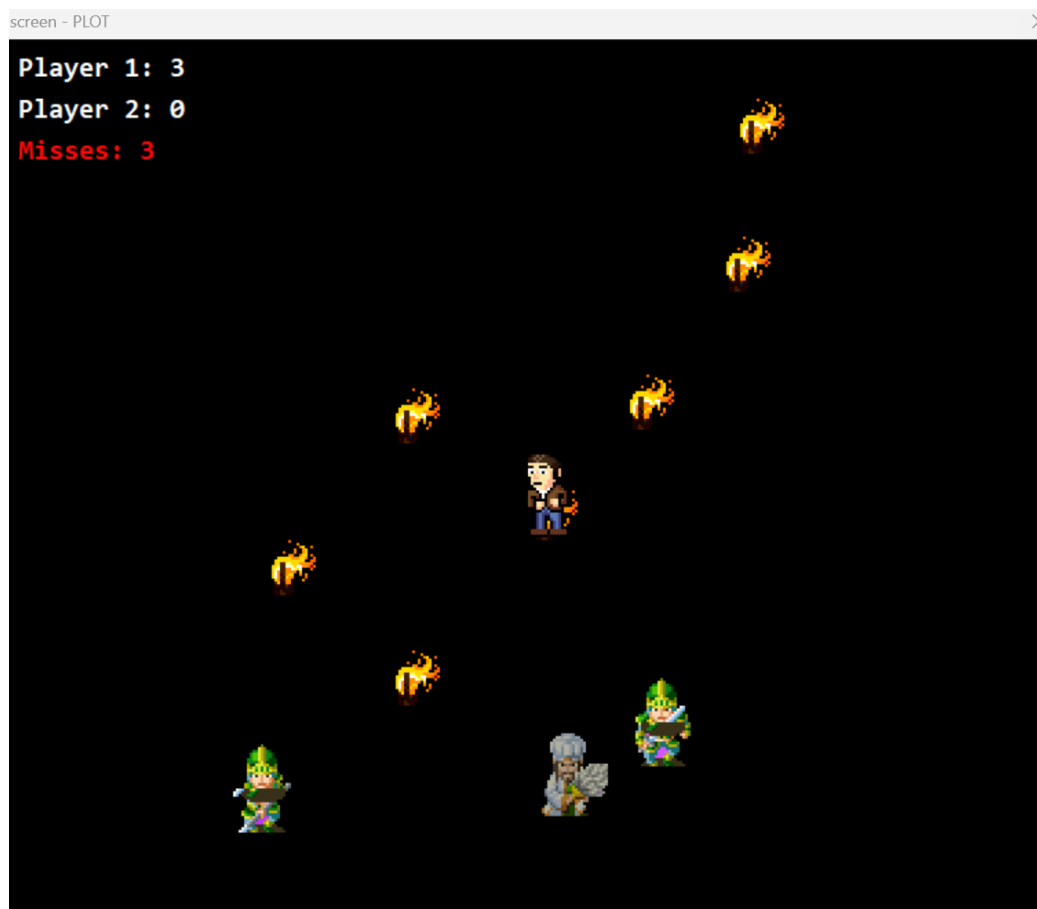
Fighting the enemies

Coding Process

The demo code provided many useful methods that we repurposed – or left entirely unchanged – for our needs. The first thing we added was a second player by adapting the code from the “WiiClassicControllerProTester_v1.0.spin2” file provided by the instructor. The next thing on our checklist was to create the “whacking” functionality – we did this using a simple “if” statement to check that the player and mole were touching while the right trigger was pressed. This was implemented inside the main loop in lines 174-185. This function was later amended to add the “whacks” counter (“Whacks”) and the attack animation. We also added a fun real-life function – when an enemy is “wacked” a signal is sent to a relay (P2) that will make a loud “CLICK!” sound every time an enemy is vanquished. This function used a variable named “collision” - we initialized this variable

believing we would use it elsewhere – it is probably unnecessary now. This can be seen in lines 187-190. The relay is not required to play the game.

Our “easter egg” function is below this one – it uses similar logic to set a variable named “EasterEgg” to true when one of the players interacts (w/ right trigger) with Zhuge Liang. You may have noticed that our “CheckCollision” functions from before are wrapped in a “repeat” - this is to ensure all seven enemies function properly. The top of this “repeat” wrapper sets the Wei sprites – before doing so, it checks that “EasterEgg” is false. The sprites of the enemies change to Prof. Garner when “EasterEgg” is true! You can now release your pent-up mechatronics-fueled rage.



The “WeiMotion()” saw many notable changes – the most notable of which being the appear-and-disappear-randomly functionality. This was accomplished using a repeat loop and the “getrnd()” function. The getrnd()” function was used to make the enemies wait a random amount of time before appearing and disappearing. It is also used to pick a random ‘hole’ (lamp) between 1-7. The rest of the loop makes the enemies rotate their sprites – this functionality was ripped from the original “WeiMotion()” code and is wrapped in a “repeatwhile” loop. This loop decrements “wait” by 175 every loop. The loop ends (and the enemy hides) once “wait” is less than or equal to zero OR the player has “whacked” the enemy.

The “GameLose()” was one of the more confusing functions to troubleshoot – our initial code was close but had some fatal logic flaws that caused it not to work. We started by hand-copying the first couple lines from “PauseGame()” and editing them for our purposes. We misplaced the “ReadController()” function before the “repeatuntil” loop(s) - we didn’t realize that this meant the function was only reading the controller inputs once before entering an endless loop. That was not the only logic flaw – the second “repeatuntil” loop initially had incorrect logic that made it such that the game did not wait for an “unpress” before restarting. We also underestimated the importance of the “StartTime:=getct()” . All of these problems – and the fact we’d been coding for four hours (exhausted!) - led to us popping the function into artificial intelligence (Gemini) to ask for help. Gemini helped straighten out the logic. We have cited this in the code accordingly. We added an animation to play in GameLose() as well as the similar function GameWin(), but

those are quite unstable. While the debug output window shows lines being executed, it is not reflected by screen change.